

Hierarchical Goal Induction

REMY OCHEI

hierarchical detection

The general system works as follows. We have a “retina” that feeds into a detection model. In the case of vision, we output the top-left and bottom-right pixel coordinates on our retina for a detected object, along with a class label. In the case of audition, operating on a mel spectrogram, we output the bounds of our detection (either a bounding box or a temporal span) together with a frequency mask, so that we know which frequencies belong to our sound.

We scale up and pad our detection, then feed it into the hierarchical detector again recursively until we produce the null detection. We also move “along” a level by conditioning on the previous detection.

the hierarchical goal inducer

Now I will describe the hierarchical goal inducer. Its behavior is simple: conditioned on the contents of its “history retina,” it outputs the start and end of any plans it sees being executed.

How do we train such a system? We can exploit the accessibility tree in macOS. As we construct the history on the retina (essentially a video), we simultaneously create a parallel JSON-based log of events. We send this log to our favorite LLM provider to detect plans and label them with descriptions. This is the level-0 training signal for our hierarchical goal inducer.

Now that we have a trained goal inducer (outputting only the temporal span of each plan for now), we can take new action trajectories and delimit goals at multiple levels of the hierarchy.

the preference model

Next, we produce a model that, conditioned on the history we have on our retina, takes in an action and outputs a probability. This is our preference model.

the action model

We attach a GPT-style head conditioned on our history and the contents of a scratchpad, and predict a payload representing our next action at each timestep. This is our first action model.

The action model is autoregressive in two dimensions: along the payload dimension and along the temporal dimension. The generation of each new payload is conditioned on the previous one.

the training loop

We train the action model through teacher forcing, then sample from the trained action model to generate data for the preference model.

Once we have a trained preference model, we can use search of all kinds to maximize the probability of an action. This constitutes a second “action” model—here we apply brute force, guided by the preference model.

To recap:

1. Train a GPT-style model to predict actions via teacher forcing.
2. Sample this model to generate training data for the preference model.
3. Apply brute-force search to maximize the preference model’s output.
4. Distill the findings via supervised learning on context–action pairs discovered through brute-force search. Here we must search for the optimal architecture; there is no way around it.

So we started with the GPT-style predictor, but we were able to level up to a leaner, meaner model. We can now replace the original GPT model with our distilled version and run the process again.

hierarchical plan detector v2.0

The first version of the hierarchical plan detector only detected temporal bounds. Now we add a GPT-style head conditioned on our history and apply it recursively, cropping the history temporally and “scaling up” in the time dimension.

We train both the bounds detector and the GPT head together.

To recap:

1. **Stage 1:** Train a plan-bounds detector that we can use hierarchically with a scale-and-pad operation.
2. **Stage 2:** Add a GPT-2-style head that we teacher-force with the cloud LLM’s outputs.
3. **Stage 3:** We no longer need the cloud LLM provider and can detect and annotate nested plans locally.
4. **Stage 4:** We fill our “actor” network’s plan scratchpad with the outputs of the distilled plan inducer. The actor’s action predictions are trained conditioned on the scratchpad’s contents.

the goal-to-action mapper

Finally, we have an agent with a trained “scratchpad”—a designated region in the network where we can inscribe a goal, and the agent will attempt to achieve it. It will not try to talk its way there; instead, it will output actions.

We have arrived at the goal-to-action mapper architecture. *QED*.